

Práctica: Histogramas y Mejora de Contraste

Objetivo

Analizar la distribución de intensidades de una imagen mediante histogramas y aplicar técnicas de mejora de contraste usando OpenCV.

Introducción al histograma

Un histograma muestra cuántos píxeles existen para cada nivel de intensidad.

- Eje X: intensidad (0–255)
- Eje Y: cantidad de píxeles

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)

hist = cv2.calcHist([img], [0], None, [256], [0,256])

plt.plot(hist)
plt.title('Histograma en escala de grises')
plt.xlabel('Nivel de intensidad')
plt.ylabel('Cantidad de píxeles')
plt.show()
```

Interpretación:

- Si el histograma está concentrado a la izquierda, la imagen es oscura.
- Si está a la derecha, la imagen es clara.
- Si está extendido, tiene buen contraste.

Actividad: Probar con una imagen oscura y otra clara y comparar sus histogramas.

Histograma de una imagen a color

En imágenes a color, cada canal tiene su propio histograma.

```
img = cv2.imread('imagen.jpg')

colores = ('b','g','r')

for i, color in enumerate(colores):
    hist = cv2.calcHist([img], [i], None, [256], [0,256])
    plt.plot(hist, color=color)

plt.title('Histograma RGB')
plt.xlabel('Nivel de intensidad')
plt.ylabel('Cantidad de píxeles')
plt.show()
```

Explicar:

- Azul : canal 0
- Verde : canal 1
- Rojo : canal 2

Actividad: Analizar qué canal predomina en una fotografía de paisaje y comparar una imagen con mucho cielo frente a una con mucha vegetación.

Aplicación de máscaras en el cálculo del histograma

A veces interesa calcular el histograma solo de una región de la imagen.

```
img = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)

mask = img.copy()
mask[:] = 0

mask[100:300,100:300] = 255

hist = cv2.calcHist([img], [0], mask, [256], [0,256])

plt.imshow(mask, cmap='gray')
plt.title('Máscara')
plt.show()

plt.plot(hist)
plt.title('Histograma de la región seleccionada')
plt.show()
```

Los píxeles blancos de la máscara se analizan y los negros se ignoran?

Actividad: Seleccionar solo el rostro o un objeto, y comparar el histograma de toda la imagen con el de la región.

Ecualización del histograma

La ecualización mejora el contraste distribuyendo mejor las intensidades.

```
img = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)

ecualizada = cv2.equalizeHist(img)

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(img, cmap='gray')
plt.title('Original')

plt.subplot(1,2,2)
plt.imshow(ecualizada, cmap='gray')
plt.title('Ecualizada')

plt.show()
```

Comparar histogramas:

```
hist1 = cv2.calcHist([img],[0],None,[256],[0,256])
hist2 = cv2.calcHist([ecualizada],[0],None,[256],[0,256])

plt.plot(hist1, label='Original')
plt.plot(hist2, label='Ecualizada')
plt.legend()
plt.show()
```

Actividad: Aplicar la ecualización a una imagen oscura y explicar si mejoró o no.

Ecualización adaptativa del histograma

La ecualización normal puede exagerar demasiado algunas zonas. Para evitarlo se usa CLAHE.

CLAHE = Contrast Limited Adaptive Histogram Equalization

```
img = cv2.imread('imagen.jpg', cv2.IMREAD_GRAYSCALE)

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

adaptativa = clahe.apply(img)

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title('Original')

plt.subplot(1,3,2)
plt.imshow(cv2.equalizeHist(img), cmap='gray')
plt.title('Ecualización normal')

plt.subplot(1,3,3)
plt.imshow(adaptativa, cmap='gray')
plt.title('CLAHE')

plt.show()
```

Actividad: Comparar la imagen original, ecualizada y adaptativa, y determinar cuál se ve mejor y por qué.